

Javascript

Async/await , error handling

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Fetch</title>
  </head>
  <body>
    <h2>Event loop</h2>

    <script>
      console.log("Start");

      setTimeout(() => {
        console.log("Middle");
      }, 0);

      console.log("End");

      // function fetchData() {
      //   fetch("https://jsonplaceholder.typicode.com/todos/1")
      //     .then((res) => {
      //       console.log(res);
      //       return res.json();
      //     })
      //     .then((data) => {
      //       console.log(data);
      //     })
      //     .catch((error) => console.log(error));
      // }
      // fetchData();

      // async await
      async function fetchData() {
        try {
          const result = await fetch(
            "https://jsonplaceholder.typicode.com/todos/1",
          );
          console.log(result);

          const data = await result.json();
          console.log(data);
        }
      }
    </script>
  </body>
</html>
```

```

    } catch (error) {
      console.log(error);
    } finally {
      console.log("Finally statement");
    }
  }

  fetchData();

  let age = 20;

  if(age > 18){
    throw new Error("Age is greater than 18")
  }

  console.log("END of the statement");
</script>
</body>
</html>

```

Theory

1. Event Loop

- **Definition:** Mechanism that allows JavaScript to handle asynchronous operations despite being single-threaded.
- **Phases:**
 - Call stack → executes synchronous code.
 - Callback queue → stores async tasks (like setTimeout).
 - Event loop → moves tasks from queue to stack when stack is empty.
- **Example:**

```

console.log("Start");
setTimeout(() => console.log("Middle"), 0);
console.log("End");

// Output: Start → End → Middle

```

2. Fetch API

- **Definition:** Modern interface for making HTTP requests.
- **Returns:** A Promise.
- **Syntax:**

```
fetch("https://jsonplaceholder.typicode.com/todos/1")  
  .then(res => res.json())  
  .then(data => console.log(data))  
  .catch(err => console.log(err));
```

3. Async/Await

- **Definition:** Syntactic sugar over Promises for cleaner code.
- **Error handling:** Use try...catch.
- **Finally block:** Executes regardless of success or failure.
- **Example:**

```
async function fetchData() {  
  try {  
    const res = await fetch("https://jsonplaceholder.typicode.com/todos/1");  
    const data = await res.json();  
    console.log(data);  
  } catch(err) {  
    console.log(err);  
  } finally {  
    console.log("Cleanup");  
  }  
}
```

4. Error Handling

- **Throwing errors:** throw new Error("message").
- **Catching errors:** try...catch.
- **Use case:** Prevents application crashes, provides meaningful feedback.



Coding Round Questions

- Event loop → Explain how the event loop works with an example.
- Fetch API → Write a program to fetch data using Fetch API.
- Async/Await → Show how async/await simplifies Promise handling.
- Error handling → Demonstrate throwing and catching errors in JavaScript.
- Promise vs async → Compare Promises and async/await.
- Finally block → Explain the role of finally in async code.



Machine Coding Round Challenge

Problem: Todo Fetcher with Error Handling

Requirements:

- Fetch todo data from <https://jsonplaceholder.typicode.com/todos>.
- Display first 5 todos in the DOM.
- Use async/await with proper error handling.
- Add a button to refetch data.
- Throw a custom error if todos are empty.